

SIM-CLR

Simple Contrastive Learning Representation

- **Problem statement:**

$\bar{x}_1, \bar{x}_2, \dots, \bar{x}_M \in D$, D is our dataset

$\bar{x}_i \in \mathbb{R}^{H \times W \times C}$ is our image

We want to learn a low dimension representation that captures all semantic features from image

$$f_\theta : \mathbb{R}^{H \times W \times C} \longrightarrow \mathbb{R}^d$$

is our representation function

$$h_i = f_\theta(\bar{x})$$

is the representation of image $\bar{x}$

- **Augmentations ($\mathcal{T}$):**

Any Transform of image, that we want to consider as the same image.

$$t_1, t_2, \dots, t_T \in \mathcal{T}$$

are the Transformations that we want to treat as acceptable.

$$t_i : \mathbb{R}^{H \times W \times C} \longrightarrow \mathbb{R}^{H \times W \times C}$$

$$i \in \{1, 2, \dots, T\}$$

- **Making Positive Pair:**

We take two randomly chosen transforms from $\mathcal{T}$ and make 2 views of image, $\tilde{x}_i = t_i(\bar{x})$ and $\tilde{x}_j = t_j(\bar{x})$. (Note: can be more but two is computationally cheap).

This forms positive pair and all other images' transforms form the negative pair.

Why is the positive made only from transforms & not the actual image?

Because, if the model was able to push representations of different allowed transforms closer then we can say that it was successful in extracting the semantics of image by using 'h'.

- **Mini batch making:**

A random batch of N images from D

$$\{\bar{x}_1, \bar{x}_2, \dots, \bar{x}_N\}$$

Augmentations $\longrightarrow$

$$\{\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_{2N}\}$$

Image $\bar{x}_k \xrightarrow{\text{+ve pair}} (\tilde{x}_{2k-1}, \tilde{x}_{2k})$

Total images = $2N$

Negatives per sample = $2(N - 1)$

- **Encoder Network:**

Can be any model according to specific task.

In paper, Res-Net 50 architecture is used.

It just maps an augmentation of an image to a representation vector.

$$h_i = f_\theta(\tilde{x}_i)$$

$$h_i \in \mathbb{R}^d$$

$$d = 2048$$

- **Projection Head:**

It is a network which maps h to space where actual contrastive loss is applied

$$z_i = g_\phi(h_i)$$

$$z_i \in \mathbb{R}^k$$

$$k = 128$$

In paper, it is taken to be a 2 layer MLP, i.e.

$$z_i = W^{(2)} \sigma(W^{(1)} h_i)$$

$$\sigma = \text{ReLU}$$

- **Similarity function:**

It is basically the normalized dot product $\rightarrow$ to ensure similar working for all images

$$\text{sim}(z_i, z_j) = \frac{z_i^T z_j}{\|z_i\| \cdot \|z_j\|}$$

This is used because we know dot product tells us how much a vector overlap with other.

- **Temperature scaling:**

We have temperature

$$\tau > 0$$

To control softmax sharpness (general stuff)

$$s_{i,j} = \frac{\text{sim}(z_i, z_j)}{\tau}$$

- **Objective:**

For a +ve pair,

Define prob that j is correct match for i

$$P_{i,j} = \frac{\exp(s_{i,j})}{\sum_{\substack{k=1 \\ k \neq i}}^{2N} \exp(s_{i,k})} \rightarrow \text{softmax}$$

From this we get loss for pair (i, j)

$$l_{i,j} = -\log P_{i,j}$$

✓

Normalized Temperature scaled Cross entropy

NT-Xent

- **Clearly $l_{i,j} \neq l_{j,i}$ (different negatives and augmentations)**

- **Final batch loss:**

$$L = \frac{1}{2N} \sum_{k=1}^N (l_{2k-1, 2k} + l_{2k, 2k-1})$$